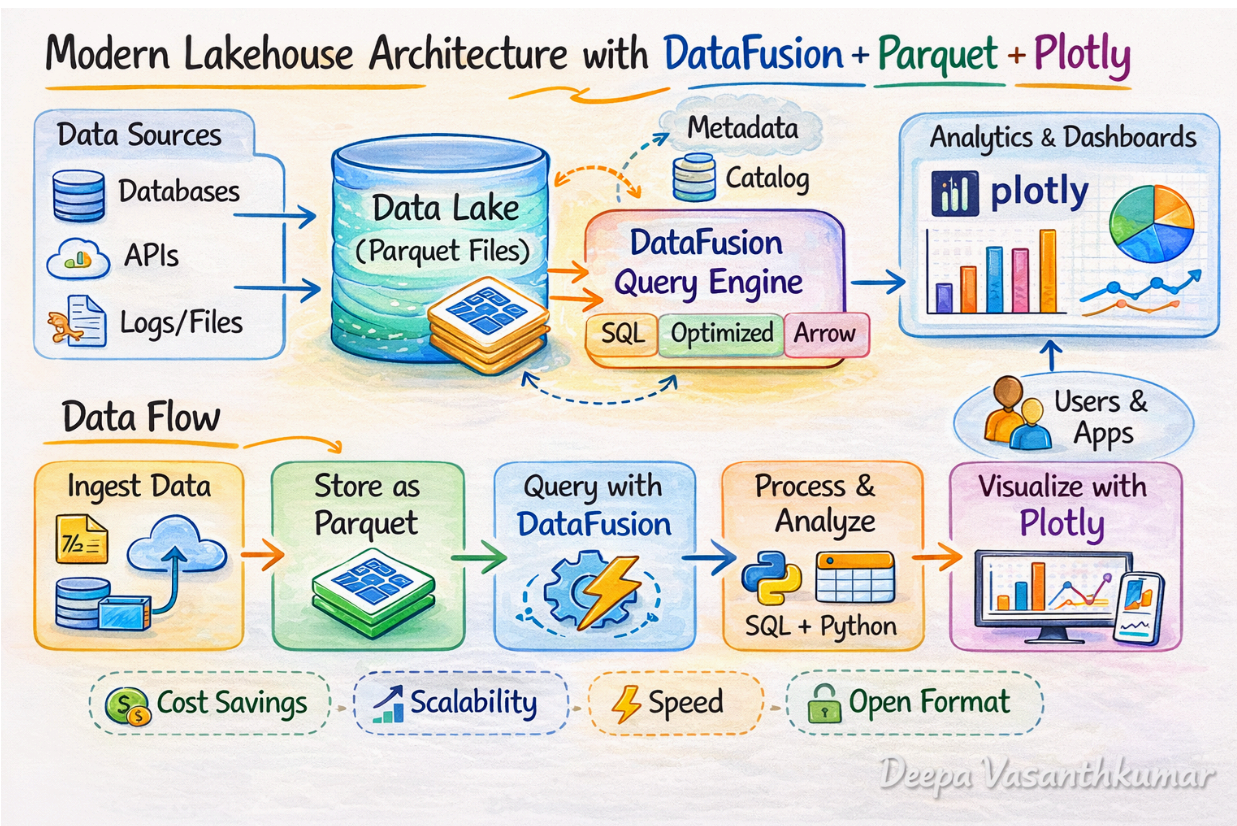
Modern Lakehouse with Datafusion + Parquet + Plotly

A lakehouse combines the flexibility of a data lake with the structured analytics capabilities of a data warehouse.

Instead of storing data in expensive proprietary databases, data is stored in open formats on object storage and queried using scalable compute engines.



Typical architecture:

This architecture enables:

- SQL analytics
- scalable storage
- interactive data exploration
- open data formats

Source code repository

https://github.com/deepavasanthkumar/modern_lakehouse_datafusion

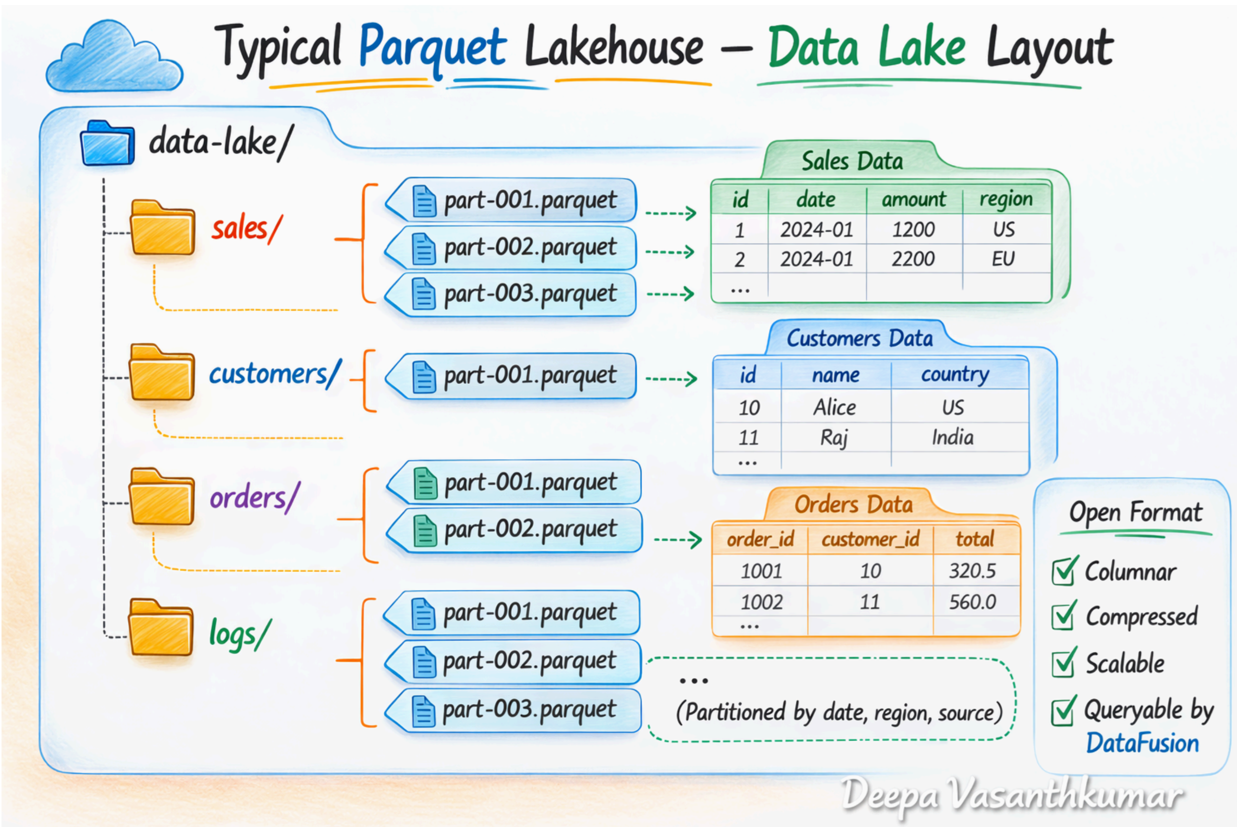
Core Components of the Stack

Storage Layer — Parquet Data Lake

The foundation of the lakehouse is **Apache Parquet**, a highly efficient columnar format designed for analytical workloads.

Benefits:

- Columnar compression
- Efficient predicate pushdown
- Reduced storage footprint
- Compatible with many engines



Because Parquet is an **open format**, data can be accessed by many engines without vendor lock-in.

Query Engine — DataFusion

The compute layer is provided by **Apache DataFusion**, a high-performance SQL query engine built in Rust.

DataFusion capabilities include:

- ANSI SQL support
- vectorized execution
- query optimization
- predicate pushdown
- Arrow-native processing

DataFusion operates directly on Parquet files without requiring a traditional database.

Example query:

Step 3 — Create DataFusion Session

```
ctx = SessionContext()
```

Step 4 — Register Data Lake DirectoryDataFusion can query multiple Parquet files as one dataset.

```
ctx.register_parquet("orders", "lake/orders/*.parquet")
```

Step 5 — Explore Dataset

```
ctx.sql("SELECT COUNT(*) as total_rows FROM orders").to_pandas()
```

	total_rows
0	2000000

Unlike heavy distributed engines, DataFusion can run **embedded applications, notebooks, or microservices**.

Analytics Layer — Plotly

For visualization, the stack uses **Plotly**.

Plotly provides:

- interactive dashboards
- web-ready visualizations
- notebook integration
- real-time chart rendering

Example analytics flow:

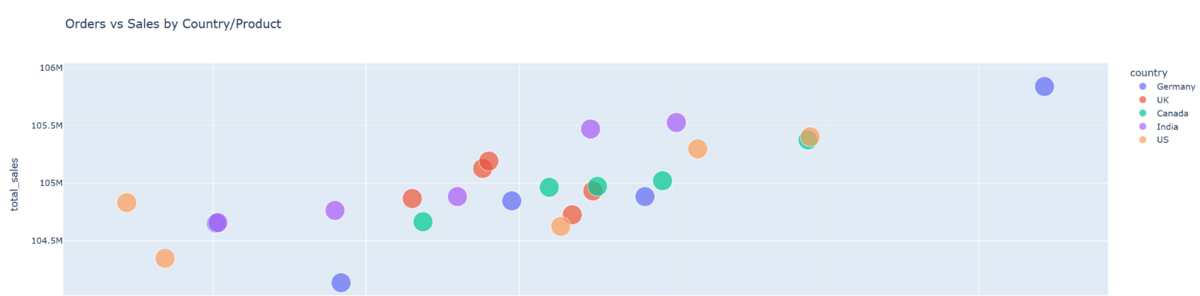
Step 8 — Advanced Analytical QueryExample of a more complex SQL query.

```
analytics = ctx.sql("""SELECT country,product,SUM(sales) as total_sales,AVG(price) as avg_price,COUNT(*) as orders FROM orders GROUP BY country, product""").to_pandas()
analytics.head()
```

	country	product	total_sales	avg_price	orders
0	Germany	Laptop	105837017	525.057445	80686
1	UK	Tablet	104933349	524.133927	80096
2	UK	Phone	104726282	523.184553	80069
3	UK	Laptop	104866854	524.765778	79860
4	Germany	Tablet	104137718	522.710168	79767

Next steps: Generate code with analytics New interactive sheet

```
fig = px.scatter(    analytics,    x="orders",    y="total_sales",    color="country",    size="avg_price",    title="Orders vs Sales by Country/Product")
fig.show()
```

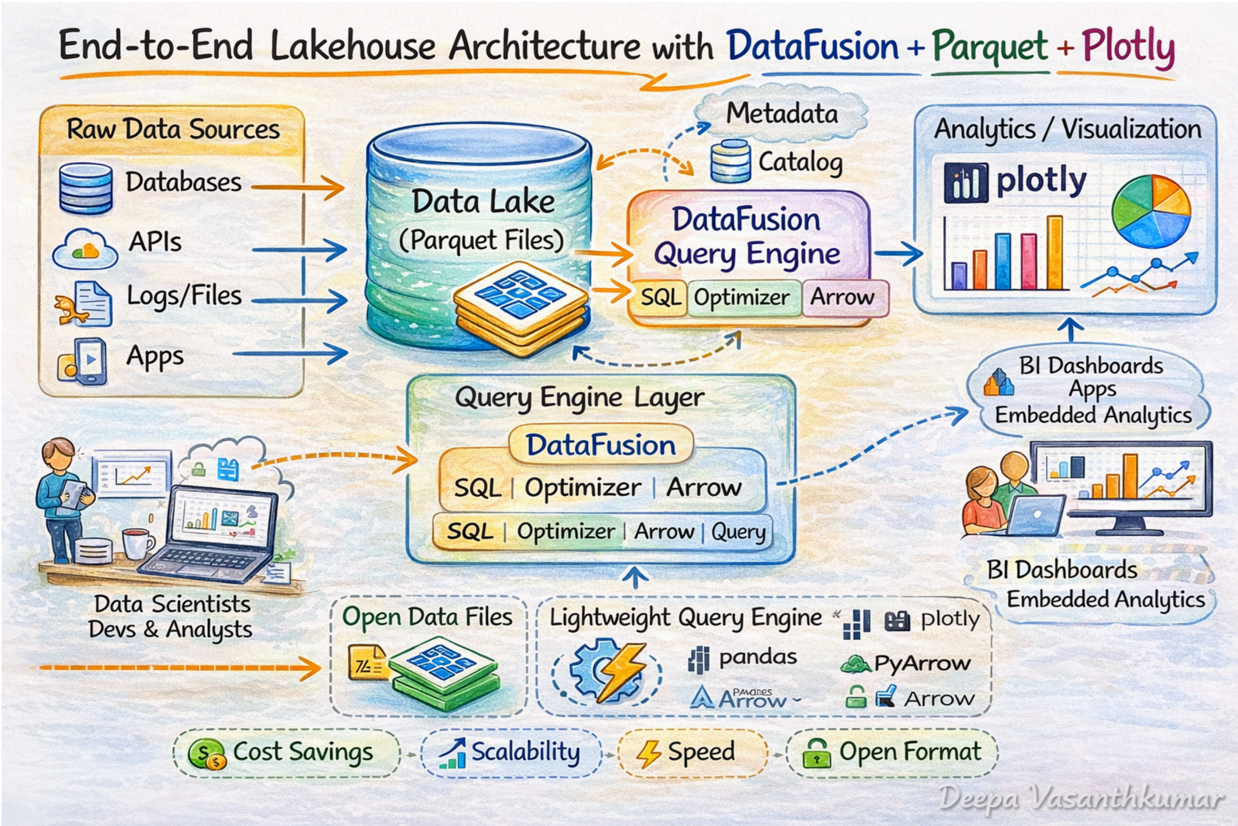


Terminal 10:29 AM Python 3

This enables rapid exploratory analytics without needing a full BI platform.

© Deepa Vasanthkumar

End-to-End Architecture



This architecture is similar to enterprise lakehouse stacks used by:

- Apache Spark
- Trino
- DuckDB

but implemented with **much lighter infrastructure**.

Cost Advantages Over Traditional Cloud Analytics

Traditional cloud analytics platforms often rely on expensive compute services such as managed clusters or proprietary warehouses.

Example platforms include:

- **Google BigQuery**
- **Amazon Redshift**
- **Snowflake**

These systems typically charge for:

- compute usage
- query execution
- data scanning
- storage
- concurrency scaling

Organizations can reduce **analytics compute costs by 70–90%** in exploratory workloads by using embedded engines.

Performance Advantages

Despite being lightweight, this architecture benefits from:

Columnar Processing

Parquet stores data by column, allowing queries to scan only relevant columns.

Example:

SELECT country FROM orders

Only the **country column** is read from disk.

Predicate Pushdown

Query filters can be pushed to the storage layer.

Example:

SELECT * FROM orders

WHERE country = 'US'

Only matching row groups are scanned.

Vectorized Execution

DataFusion processes data in batches using the **Apache Arrow memory format**, improving CPU efficiency.

Ideal Use Cases

This stack works particularly well for:

- Data exploration notebooks
- Embedded analytics in applications
- Small to medium analytics workloads
- Edge analytics environments
- Data engineering prototypes

It can also serve as a **local analytics engine for data scientists** working with data lakes.

Benefits for Data Engineering Teams

Key advantages of this modern lakehouse stack include:

- open data formats
- minimal infrastructure overhead
- vendor neutrality
- lower operational cost
- rapid experimentation

It allows engineers to build analytics pipelines without deploying large clusters.

Future Evolution

This architecture can evolve into a full enterprise lakehouse by adding:

- table formats such as **Apache Iceberg**
- orchestration tools
- metadata catalogs
- governance frameworks

At that stage, the stack resembles modern lakehouse architectures used in large organizations.